DDD workshop PXL 2026

Dag 2

Dag 2 — Van aggregate tot werkende applicatie

Herhaling dag 1

Value objects

Validatie van objecten; compare by value; immutable

Entity

Mutable; heeft een Id

Aggregates

Commands uitvoeren op de aggregates; check domain rules; voeren domain events uit op de state; enige startpunt van onze domain objecten — deze bewaakt de consistentie

Domain services

Bevat domain logica die niet in de aggregate past, of herhaald wordt op meerdere plaatsen; kan geïnjecteerd worden met DI

Onion architecture

Domain staat centraal

Ubiquitous language

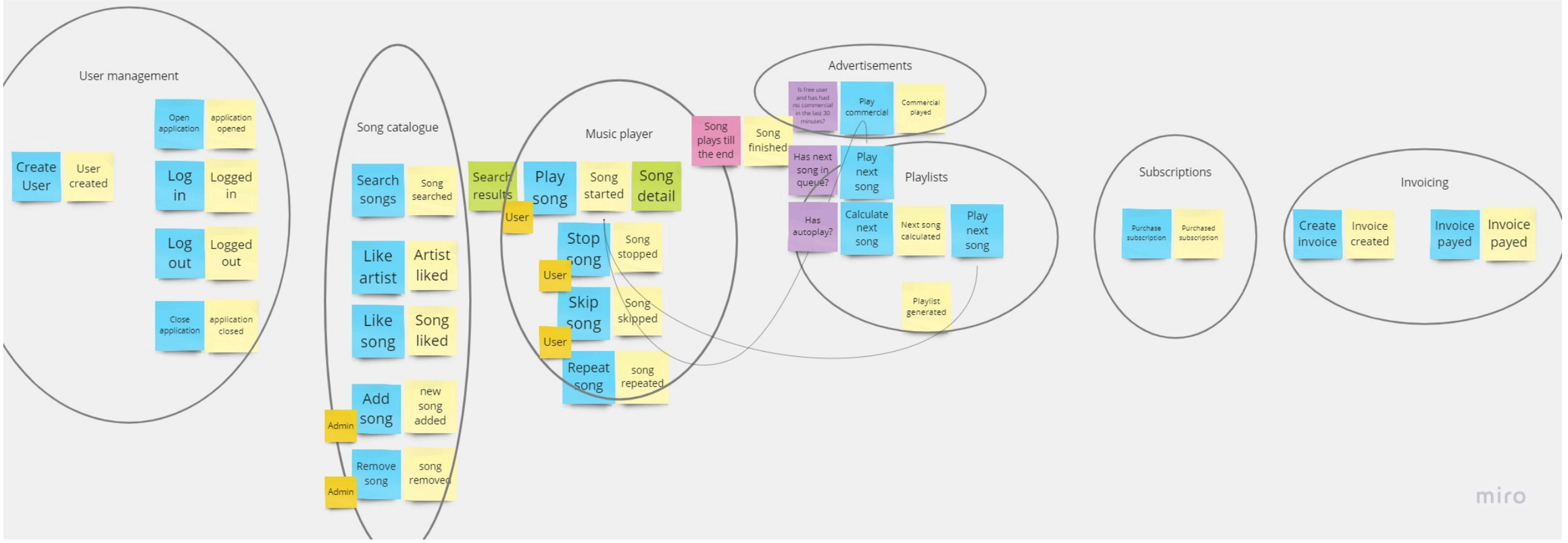
Tactical vs Strategic

Strategic DDD kwam in de vorige sessie aan bod; vandaag werken we de tactical kant uit.

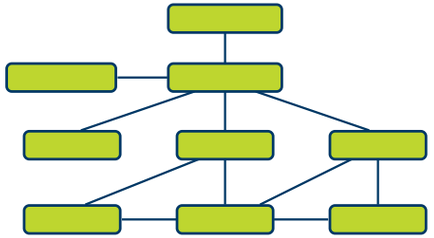


Strategic — voorbeeldbord

Zelfde type bord als in de strategic-sessie — oranje (commands) en blauw (events) domineren het bord: wat je daar verzamelde, wordt vandaag een aggregate met commands en events.



Tactical — bouwstenen (recap)



Bouwsteen	Kern	Kenmerken
Value object	Beschrijvend aspect, geen identiteit	immutable · compare by value · simpel design
Entity	Continuïteit door een levenscyclus	mutable · unieke id binnen het domein
Aggregate	Cluster objecten als één eenheid voor datawijzigingen	exact 1 root · bewaakt consistency · enige toegang voor changes
Factory	Constructie van complexe objectgrafen	bij te complexe creatie · gebruikt vaak DI
Repository	Alle objecten van een type als querybare collectie	houdt entities bij · database, cache, text file, …
Domain service	Proces/transformatie buiten entity en VO	encapsuleert domeinlogica · haalt info op voor validatie
Architectuur	Onion / hexagonal	Domain Core centraal · afhankelijkheden naar binnen · interfaces

Herhaling — onion architecture

Domain staat centraal

Domain logica staat centraal

Naar binnen

Afhankelijkheid is naar binnen toe (inversion of control); domain logica is niet afhankelijk van de infrastructuur

Via interfaces

Alles is losgekoppeld via interfaces, beschreven in de application- en domain layer

Herhaling — application service

- Use case oriented — gefocust op het vervullen van een business use case
- Bevat **geen** domain logica
- Allerlaatste moment waarop er geconverteerd wordt naar domain types
- Beschrijft een proces; voornamelijk stateless

Typisch voltooit een application service de volgende stappen:

1. Validatie van de input
2. Ophalen van domain entities
3. Uitvoeren van domain commands op een aggregate
4. Opslaan van domain entities
5. Notificaties verzenden van belangrijke events

DAG 2 · LAB 6

Oefening 6

- Structureer per aggregate: `Domain` en `Application`
- Werk de application service `AlbumService` uit (stub staat klaar)
- Koppel de api-endpoints via DI aan de service
- Maak van buiten het domein een album aan en wijzig de beschrijving

Verdieping van oefening 5 van dag 1 (dag 1 → oefening 5) — een in-memory repository volstaat.

15 min

STARTER

`Dag 2/Starter`

Klaar wanneer: je via de api/console een album kan aanmaken en de beschrijving kan wijzigen, met het domein als enige drager van domeinlogica.

Persistence

- Data opslaan in een database
- Laten we starten van de code.

State stored vs event sourcing

State stored

- Sla de state op in de database: voeg **id** toe, voeg **version** toe — version staat voor **optimistic concurrency**: een gelijktijdige tweede schrijver botst op de versie en verliest (of herlaadt en probeert opnieuw)
- Vervuil je domain state niet met een constructor

Event sourcing

- Sla enkel de **events** op in de database
- Bouw de aggregate elke keer terug op (replay van de event stream)
- ⚠️ Zorg dat je steeds backwards compatible blijft

Event sourcing — praktisch

Expected version

Append je events mét de verwachte streamversie (`Version` op je aggregate); een conflict = iemand schreef eerst → herlaad en probeer opnieuw.

Snapshots

Bij veel events bewaar je periodiek een state-snapshot zodat de heropbouw beperkt blijft.

Upcasting

"Backwards compatible" concreet gemaakt — upcasters zetten oude event-versies om naar het huidige schema zodat de stream leesbaar blijft.

Kosten

Event sourcing brengt eventual consistency en extra tooling/operatie mee — weegt niet altijd op, vandaar "Anders hou je de state bij".

Wat kies je?

- **Event sourcing** wanneer:
  - je historische data moet bijhouden
  - je wil bijhouden **waarom** de state is veranderd
  - je in staat wil zijn om de state terug op te bouwen op eender welk moment in het verleden
- Anders hou je de **state** bij
- **Hybrid**: state voor het hoofdgebruik, event stream als audit trail

DAG 2 · LAB 7

Oefening 7

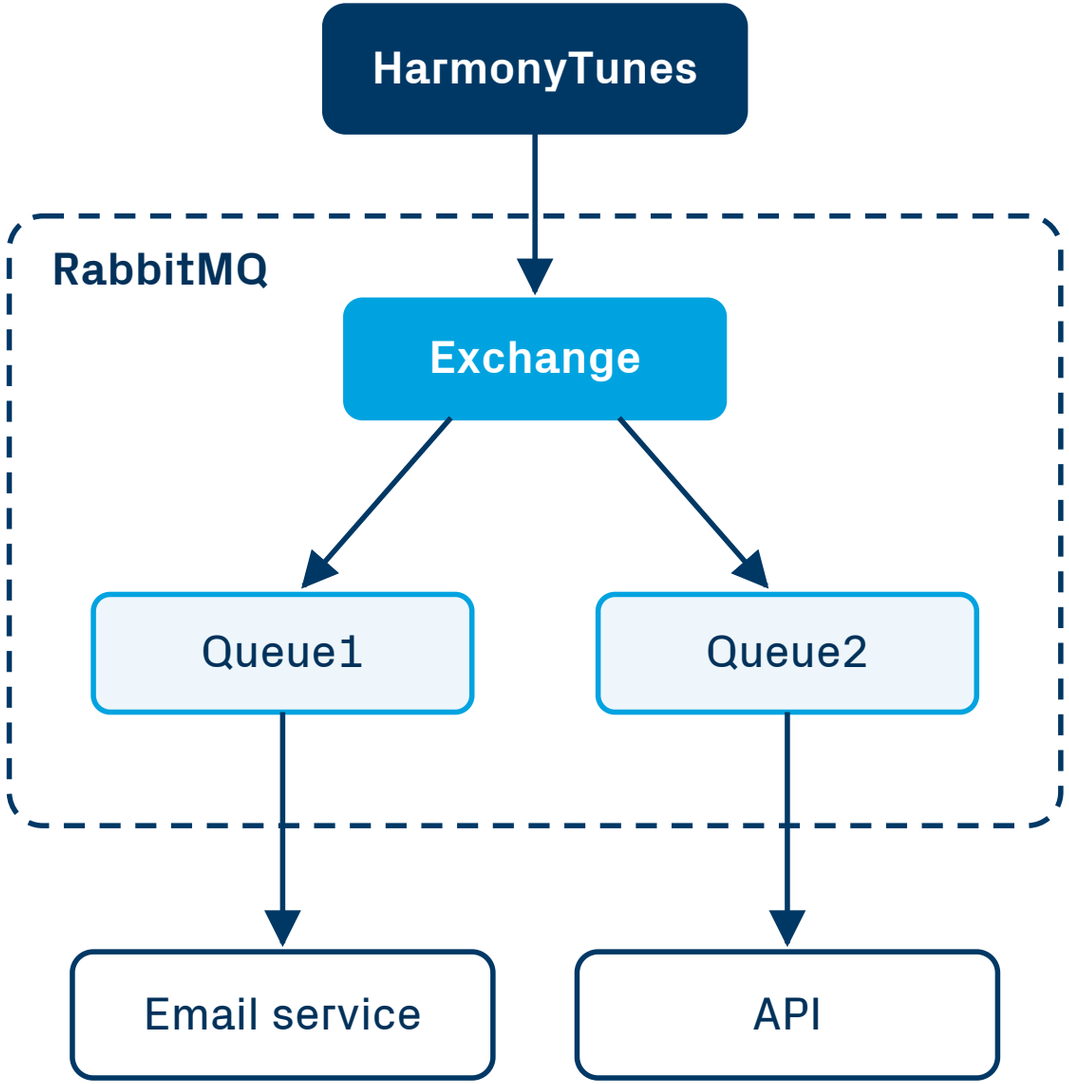
- Implementeer de database
- Sla je state op in de database
- Kies zelf voor event sourced of state stored

STARTER

`Dag 2/Starter` — verder dan lab 6

Klaar wanneer: na een herstart van je applicatie de aggregate opnieuw kan geladen worden uit je keuze (state of event stream).

Messaging



- Niet essentieel voor DDD maar veel gebruikt
- Twee soorten events:
  - **Commands**: er moet een actie uitgevoerd worden
  - **Notifications**: er is iets gebeurd

Twee soorten events

command

Command events

- Een domain beslist dat een actie moet gebeuren.
- Dit event heeft een specifieke ontvanger in gedachten.
- Dit event veroorzaakt typisch iets in een ander domain.

notification

Notification events

- Een domain deelt mee dat iets gebeurd is.
- Dit event wordt breed uitgestuurd.
- Kan 0, 1 of meerdere ontvangers hebben.
- Bevat: enkel een id van de verzender; of alle informatie relevant aan de notificatie.

Idempotente handlers

> Asynchrone bezorging kan duplicaten geven — je handler kan hetzelfde bericht meerdere keren krijgen. Zorg dat verwerken twee keer hetzelfde resultaat geeft als één keer: houd verwerkte bericht-id's bij, of werk idempotent (upsert i.p.v. insert).

Synchroon vs asynchroon

Synchroon

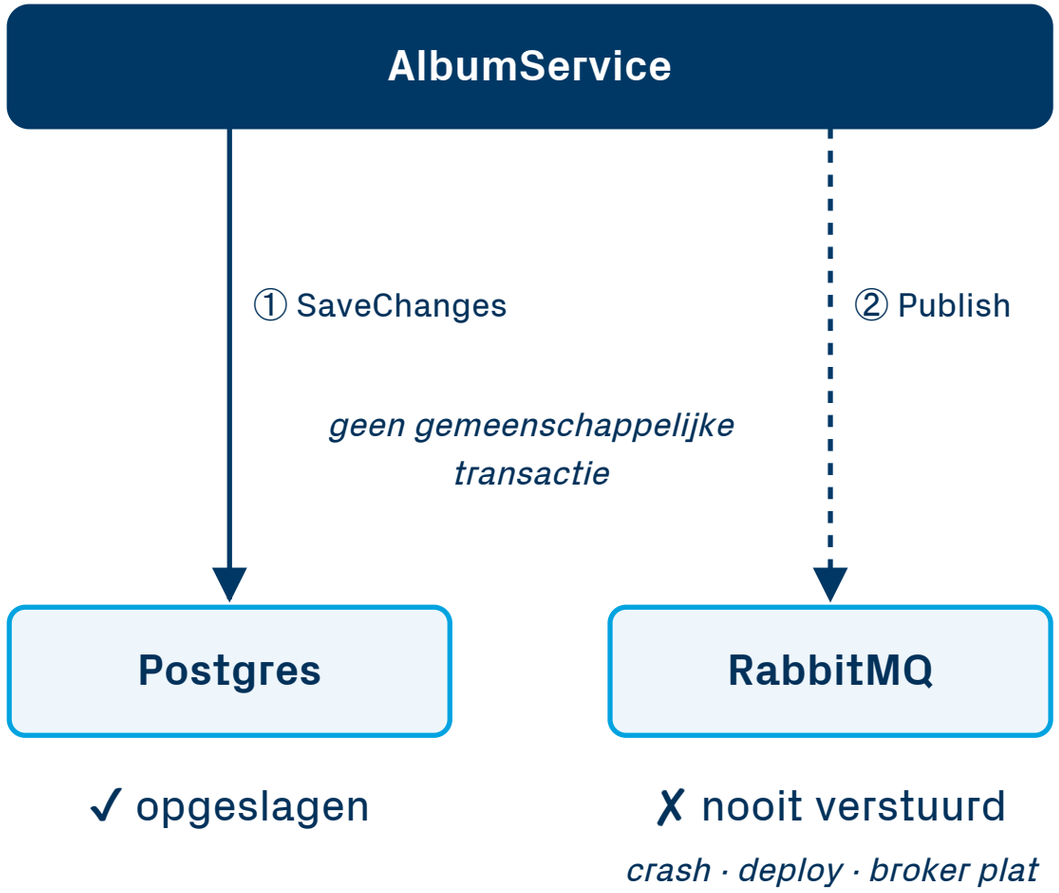
- Alles in 1 proces
- Resultaat naar gebruiker als alles klaar is
- Kan transactioneel

Asynchroon

- Message bus is vereist
- Kan niet transactioneel — vereist het **Saga**-patroon (reeks lokale transacties met compensatiestappen)
- Robust
- Eventual consistency

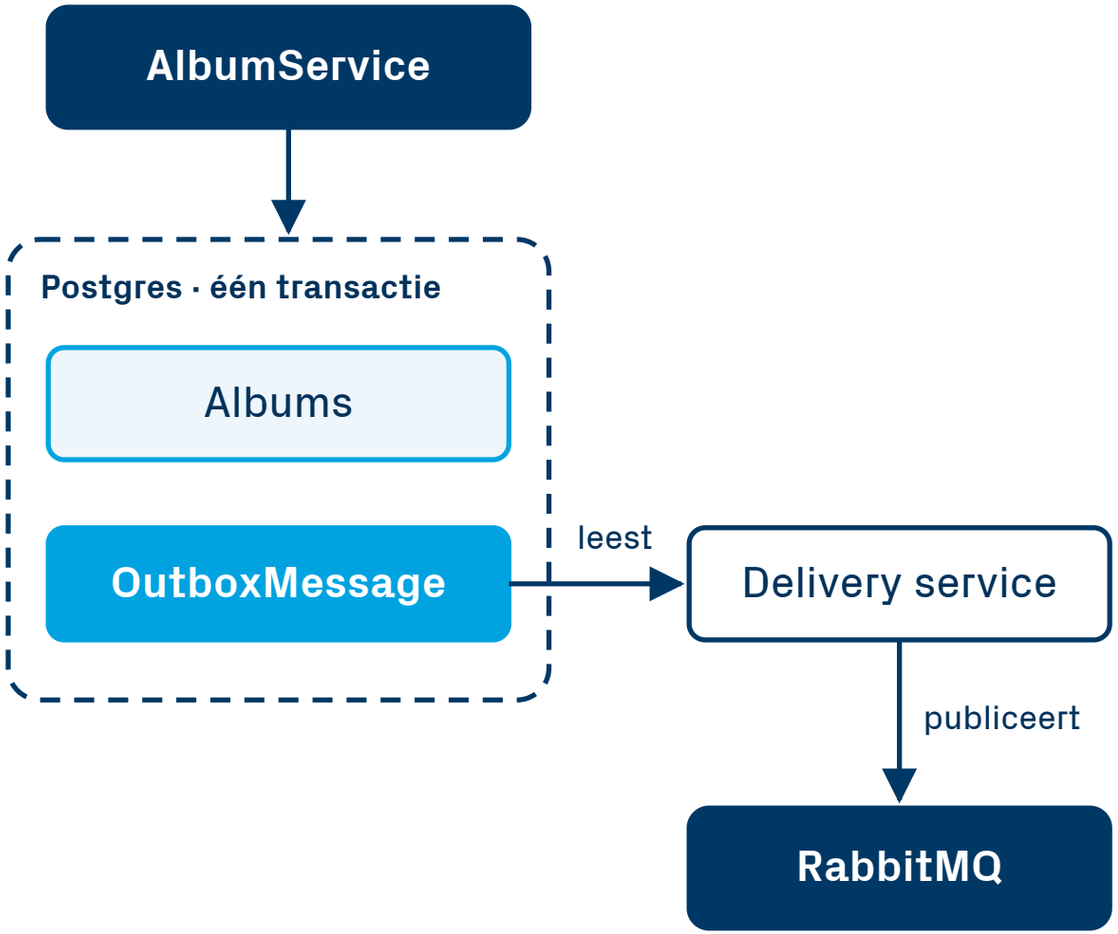
Het probleem: dual write

Publiceren ná het opslaan: twee systemen, geen gemeenschappelijke transactie.



- Commit gelukt, publish mislukt → het album bestaat, maar geen enkele handler hoort ervan
- Eerst publish, dan commit? Commit mislukt → handlers reageren op iets dat nooit gebeurd is
- Het gebeurt **stil**: de gebruiker krijgt gewoon een 200

Outbox pattern



- Schrijf het bericht naar **dezelfde database, in dezelfde transactie** als je state
- Een **delivery service** (relay) verstuurt het later naar de broker en verwijdert de rij
- Alles of niets: state én bericht, of geen van beide
- Broker plat? De berichten wachten in de outbox tot hij terug is
- Levering blijft **at least once** → idempotente handlers blijven nodig

Outbox met MassTransit

```
// RegisterMessenger
configure.AddEntityFrameworkOutbox<CatalogueContext>(outbox =>
{
    outbox.UsePostgres();   // lock-SQL voor Postgres (default: SQL Server)
    outbox.UseBusOutbox();  // IPublishEndpoint schrijft naar de outbox
});

// CatalogueContext.OnModelCreating
modelBuilder.AddInboxStateEntity();
modelBuilder.AddOutboxMessageEntity();
modelBuilder.AddOutboxStateEntity();
```

- Package `MassTransit.EntityFrameworkCore` + een migratie: tabellen `OutboxMessage`, `OutboxState` en `InboxState`
- Enkel de scoped `IPublishEndpoint` gaat via de outbox; `IBus.Publish` niet
- Het domein verandert niet: alles zit in Ports

De volgorde telt

```
public async Task PublishAlbum(AlbumReference albumReference)
{
    await Update(albumReference, async agg => {
        agg.Publish();

        await _messagePublisher.Publish(new AlbumUpdatedNotification
        {
            AlbumReference = agg.Id.Key
        });
    });   // Update commit: aggregate + bericht in één transactie
}
```

- `Publish` zet het bericht enkel klaar; pas de commit (`SaveChanges`) bewaart het
- Publiceer daarom binnen de actie van `Add` / `Update`: hun commit bewaart aggregate én bericht
- Publish ná `Add` / `Update` → de commit is al gebeurd: het bericht wordt nooit opgeslagen, en ook dat faalt stil
- Stap 4 en 5 van de application service worden één transactie

Demo — trek de stekker eruit

1. Wat gebeurt er als RabbitMQ plat ligt?
2. `docker stop harmonytunes-rabbitmq`
3. Maak een album aan en publiceer het → de API antwoordt gewoon
4. Toon `"Catalogue"."OutboxMessage"` in pgAdmin: de berichten wachten
5. `docker start harmonytunes-rabbitmq` → de rijen verdwijnen en de handler verwerkt de berichten

DAG 2 · LAB 8

Hands on — Oefening 8

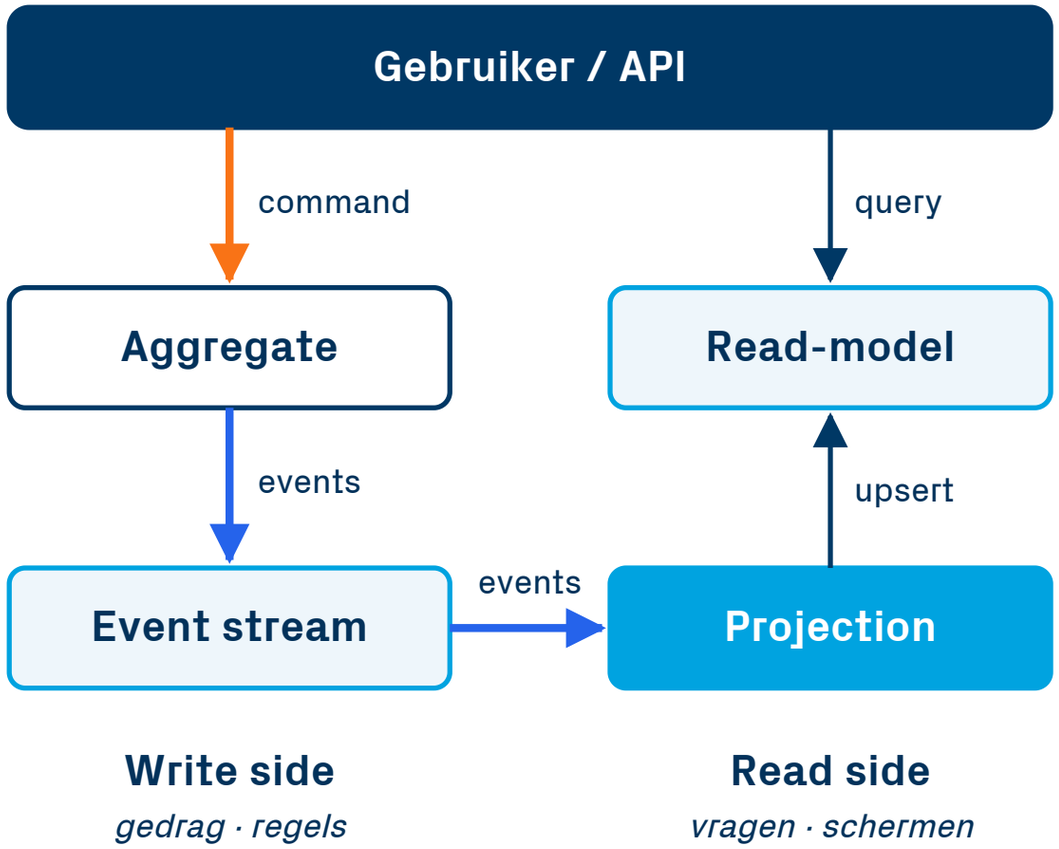
- Implementeer een side effect in je code
- Verstuur een command/notification
- Behandel het event
- Verstuur via de outbox: aggregate en bericht in één transactie

STARTER

`Dag 2/Starter` — verder dan lab 7

Klaar wanneer: bij het opslaan van je aggregate een bericht via de outbox naar buiten gaat en een handler dat effectief ontvangt en verwerkt, ook als RabbitMQ even plat lag.

Projections



- Een **projection** is een read-model, gebouwd voor **één vraag**: een scherm, een lijst, een rapport
- Wordt bijgewerkt door events: elke gebeurtenis → een update van het read-model
- Plat en gedenormaliseerd: geen value objects, geen gedrag, geen domain rules
- Koppelt je aggregate los van je query-modellen
- Het splitsen van je gedrag en queries (**CQRS**)
- Handig wanneer je met event sourcing werkt, maar ook bruikbaar bij state stored

CQRS — twee modellen

command

Command side

- De aggregate: één model dat de consistentie bewaakt
- Domain rules, invarianten en gedrag
- Geoptimaliseerd voor schrijven: laden per id, één aggregate per transactie
- De bron van de waarheid

query

Query side

- Projections: zoveel read-modellen als je vragen hebt
- Geen domeinlogica: plat, gedenormaliseerd, klaar om te tonen
- Geoptimaliseerd voor lezen: filteren, sorteren en joinen al vooraf gedaan
- Afgeleide data → eventual consistent; mag je weggooien en herbouwen

CQRS = Command Query Responsibility Segregation (Greg Young), een uitbreiding van CQS (Bertrand Meyer): een methode verandert state óf geeft data terug, nooit beide.

Geen replay bij elke read

Zonder projection



bij élke read: alle events laden en opnieuw toepassen

Met projection



de projection deed het werk al toen het event binnenkwam

- Een event sourced aggregate lezen = de stream laden en **alle events opnieuw toepassen**
- Lijstscherm: 1.000 albums × 50 events = 50.000 events per request
- "Alle albums van artiest X?" Een event store laadt enkel per stream-id
- Snapshots helpen voor één aggregate, niet voor een lijst of een zoekvraag
- Met een projection gebeurt de replay **één keer**, wanneer het event binnenkomt; lezen wordt één `SELECT`

Use cases uit de praktijk

Overzichtsschermen

Catalogus met gepubliceerde albums, hun artiest en het aantal songs. Data uit Album, Song én Artist, vooraf samengevoegd in één rij.

Zoeken

Songs en artiesten in een zoekindex (Elasticsearch, Postgres full-text): zoeken op titel of genre, iets wat je niet per id kan laden.

Saldo & tellers

Je banksaldo is een projection van al je transacties. Zo ook: streams per album, voorraad per product — vooraf opgeteld.

Dashboards & rapporten

Omzet per dag, top 10 van de week: een read-model per rapport, eventueel in een aparte reporting-database.

Andere bounded context

Bv. Billing houdt een lokale kopie bij van de albumprijzen uit Catalogue: geen synchrone call, blijft werken als Catalogue plat ligt.

Nieuwe vraag, oude data

Met event sourcing bouw je achteraf een nieuwe projection door alle events opnieuw af te spelen: inzicht in data van vóór de feature.

Projections — aandachtspunten

Eventual consistency

Het read-model loopt even achter op de aggregate. Toon na een command het resultaat van dat command, geen verse query (read your own writes).

Idempotent

Hetzelfde event twee keer verwerken = hetzelfde read-model. Upsert i.p.v. insert.

Volgorde

Berichten kunnen in de verkeerde volgorde binnenkomen. Bewaar een versie per rij en negeer oudere updates, of herlees de actuele state (zoals in HarmonyTunes).

Herbouwbaar

Een projection is afgeleide data: weggooien en opnieuw opbouwen mag altijd. Nieuw schema = nieuwe projection + replay.

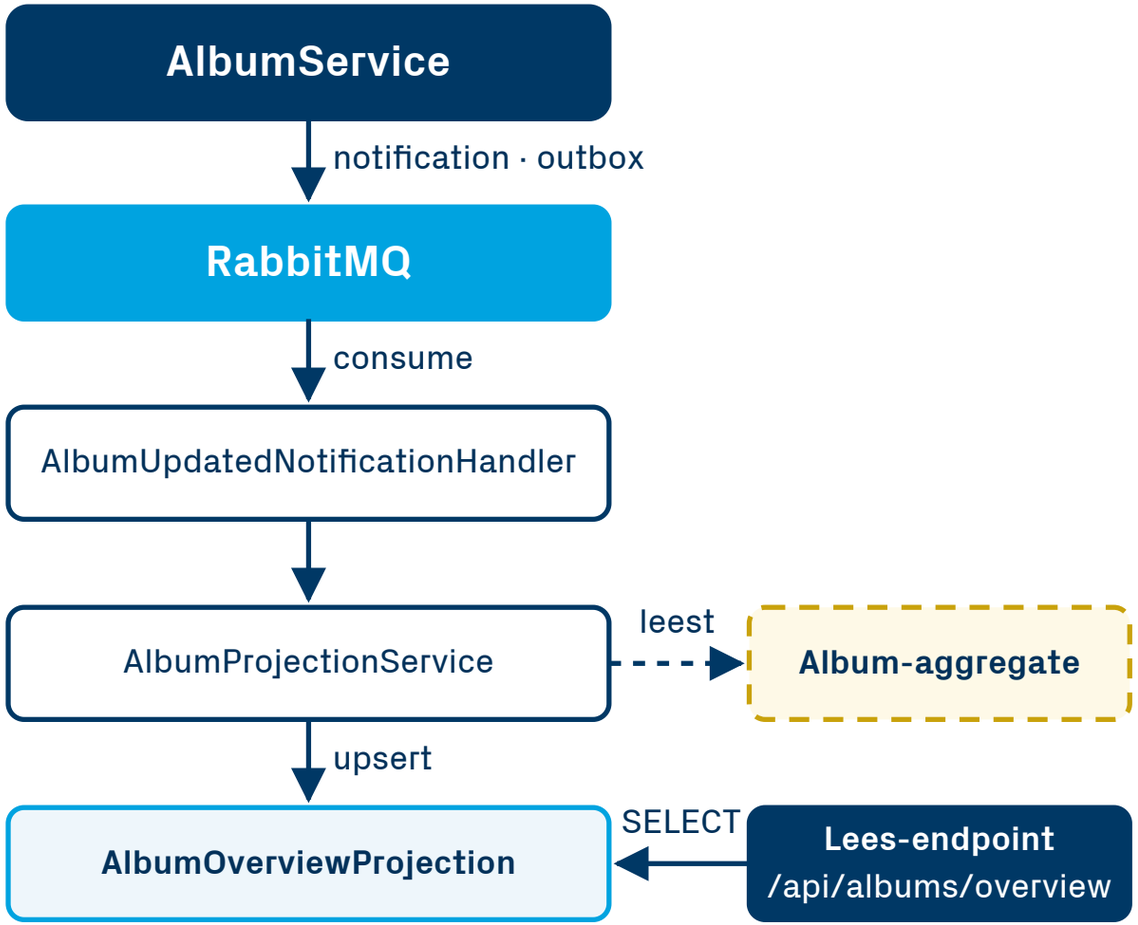
Eén per vraag

Bouw geen generiek read-model dat alles kan. Elke vraag mag zijn eigen model en opslag hebben (tabel, Redis, zoekindex).

Kosten

Extra code, extra opslag, een bus om te bewaken. Eén simpel scherm op een state stored aggregate? Lees dan gewoon de state.

Projections in HarmonyTunes



- Je bouwt verder op lab 8: de handler die het bericht ontvangt, werkt nu de projection bij
- `AlbumOverviewProjection`: een platte klasse met enkel wat het overzichtsscherm nodig heeft
- De projection service mag de aggregate lezen om de rij op te bouwen
- Het lees-endpoint leest **enkel** de projection: geen repository, geen aggregate
- Na een update klopt het overzicht na een korte vertraging (eventual consistency)

DAG 2 · LAB 9

Oefening 9

- Implementeer een projection service
- Query data via projections i.p.v. aggregates

STARTER

`Dag 2/Starter` — verder dan lab 8

Klaar wanneer: je query-flow via de projection loopt en je aggregate niet meer aangesproken wordt voor lezen.

Cheat sheet

Bouwstenen (kernpunten van beide dagen):

Bouwsteen	Verantwoordelijkheid
Value object	Beschrijvend aspect zonder identiteit; immutable, compare by value
Entity	Continuïteit door een levenscyclus; unieke id
Aggregate	Eén consistentiegrens met één root; commands → rules → events
Factory	Complexe creatie
Repository	Aggregate roots bijhouden, per root
Domain service	Domeinlogica buiten entity/VO
Application service	Use case orkestratie zonder domeinlogica
Projection / CQRS	Query-modellen gescheiden van aggregate

Beslissingslijst

Event sourcing vs state stored

ES bij historische data, "waarom"-vragen en tijdreizen; anders state; hybrid voor audit trail.

Synchroon vs asynchroon

Sync binnen één transactie/resultaat-vraag; async via bus met outbox, saga + idempotente handlers en eventual consistency.

Command vs notification

Command: specifieke ontvanger, actie in ander domein; notification: breed uitgestuurd, 0..n ontvangers.

Referenties

- Eric Evans — *Domain-Driven Design: Tackling Complexity in the Heart of Software* (2003)
- Vaughn Vernon — *Implementing Domain-Driven Design* (2013); *Domain-Driven Design Distilled* (2016)
- Alberto Brandolini & Enrico Campidoglio — *Introducing Event Storming* (2018)
- Milo Millet — *Getting Started with Domain-Driven Design* (2015)
- Code + slides: https://bit.ly/PXL-DDD-2025

Info

Leuke programmeerwedstrijd op 24 oktober:
CodeBash 2026

QR-code — CodeBash

Nog op zoek naar een leuke, creatieve stage-opdracht? Kom even praten tijdens de handshake event.

QR-code — stage opdrachten